

SQL Cheatsheet — 1-Page Reference

Print it. Pin it. Walk in confident.

Section A — 5 Most-Asked Function Patterns

1. ROW_NUMBER() — Dedupe / Keep Latest Per Group

Use it when you need the most recent row per user/order/session and want to drop the rest.

```
SELECT * FROM (
  SELECT o.*,
         ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY created_at DESC) AS rn
  FROM orders o
) t
WHERE rn = 1;
```

2. SUM() OVER — Running Totals / Cumulative Aggregates

Use it when you need a running balance, cumulative revenue, or rolling spend per customer.

```
SELECT payment_id, user_id, amount,
       SUM(amount) OVER (PARTITION BY user_id ORDER BY paid_at
                        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_total
FROM payments;
```

3. LAG / LEAD — Period-Over-Period Comparisons

Use it when comparing this month vs last month, or current order value vs the previous one.

```
SELECT user_id, txn_date, amount,
       LAG(amount, 1) OVER (PARTITION BY user_id ORDER BY txn_date) AS prev_amount,
       amount - LAG(amount, 1) OVER (PARTITION BY user_id ORDER BY txn_date) AS delta
FROM transactions;
```

4. CASE Inside an Aggregate — Conditional Aggregation

Use it when you need a pivot-style summary (paid vs refunded, COD vs prepaid) in one pass.

```
SELECT user_id,
       SUM(CASE WHEN status = 'paid' THEN amount END) AS paid_total,
       SUM(CASE WHEN status = 'refunded' THEN amount END) AS refunded_total,
       COUNT(CASE WHEN status = 'failed' THEN 1 END) AS failed_count
FROM payments
GROUP BY user_id;
```

5. COALESCE & NULLIF — NULL Handling Idioms

Use COALESCE for fallback chains; use NULLIF to convert sentinel values (or avoid divide-by-zero).

```
SELECT user_id,
       COALESCE(billing_phone, contact_phone, 'NA') AS phone,
       amount / NULLIF(quantity, 0) AS unit_price
FROM orders;
```

Section B — 8 Common Index Pitfalls

1. Function on Indexed Column

`WHERE UPPER(name) = 'AMIT'` forces a full scan — the B-tree is on `name`, not `UPPER(name)`. **Fix:** store a normalized column, or create an expression index — Postgres: `CREATE INDEX idx_users_uname ON users (UPPER(name));`

2. Implicit Type Cast

`WHERE int_id = '5'` (string literal vs int column) silently disables the index on many engines. **Fix:** match types exactly — pass `5` (integer) for an integer column, `'5'` for a varchar.

3. Leading-Wildcard LIKE

`WHERE col LIKE '%foo'` cannot use a standard B-tree — only a trailing wildcard (`'foo%'`) can. **Fix:** trigram index in Postgres (`CREATE EXTENSION pg_trgm; CREATE INDEX ... USING gin(col gin_trgm_ops);`) or a full-text search index.

4. Low-Cardinality Column

A standalone index on `is_active` , `gender` , or `status` is often ignored — too few distinct values. **Fix:** composite index leading with a selective column (e.g., `(user_id, is_active)`) or a **partial index** (`WHERE is_active = true`).

5. OR Across Different Columns

`WHERE a = 1 OR b = 2` frequently falls back to a table scan even if both columns are individually indexed. **Fix:** rewrite as `UNION` of two indexed queries, or build a composite index if the access pattern is fixed.

6. Composite Index Column-Order Mismatch

An index on `(a, b)` does NOT help `WHERE b = ...` alone — the leading column must appear in the predicate. **Fix:** lead with the column you filter on most often; add a separate index on `b` if needed.

7. NOT IN / NOT EXISTS with NULLs

`WHERE id NOT IN (SELECT user_id FROM blacklist)` returns **zero rows** if even one `user_id` is NULL. **Fix:** use `NOT EXISTS (... WHERE blacklist.user_id = users.id)` , or filter `WHERE user_id IS NOT NULL` inside the subquery.

8. Stale Statistics

The planner picks bad plans (wrong join order, wrong index) when row-count estimates are out of date. **Fix:** Postgres: `ANALYZE table_name;` · MySQL: `ANALYZE TABLE table_name;` · SQL Server: `UPDATE STATISTICS table_name;`

Section C — 3 Transaction Isolation Tables

Table C1 — Isolation Levels vs Anomalies

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read
Read Uncommitted	Allowed	Allowed	Allowed
Read Committed	Prevented	Allowed	Allowed
Repeatable Read	Prevented	Prevented	Allowed (prevented in MySQL InnoDB via gap locks)
Serializable	Prevented	Prevented	Prevented

Table C2 — Engine Defaults

Engine	Default Isolation Level
PostgreSQL	Read Committed
MySQL (InnoDB)	Repeatable Read
SQL Server	Read Committed
Oracle	Read Committed
Snowflake	Read Committed
BigQuery	Snapshot (per query)

Table C3 — Lock Mode Compatibility

Mode held → / Mode requested ↓	Shared (S)	Update (U)	Exclusive (X)
Shared (S)	compatible	compatible	blocks
Update (U)	compatible	blocks	blocks
Exclusive (X)	blocks	blocks	blocks

Intent locks (IS / IX / SIX) sit at the table/page level to signal that finer-grained row locks of the corresponding type exist below. They make granular locking efficient — an exclusive table lock can immediately see incompatible intent locks instead of scanning every row.